Planet-Scale URL Shortener: High-Level Design Thesis

Document Classification: Principal-Level System Design — Interview Reference & Production Architecture Guide
Author: Infrastructure Architecture Division
Date: October 2026
Version: 1.0

Table of Contents

1. Interview Framing & Scope Bounding
2. Clarifying Questions & Core Assumptions
3. Measurable Outcomes (Functional & Non-Functional)
4. Workload-Specific Goals & Capacity Reasoning
5. CAP Theorem & Boundary Choice
6. Domain Model & Database Invariants
7. API Contract Design
8. Token Generation Service (TGS) Deep Dive
9. Core Architecture: First Pass to Final Shape
10. Lifecycle & Concurrency Mechanics
11. Traffic Surge & Cache Stampede Mitigation
12. Partitioning & Data Sharding Strategy
13. Database & Distributed Caching Layer Layout
14. Failure Behavior & Operational Signals
15. Interview Walkthrough Decision Summary & Checklist

1. Interview Framing & Scope Bounding

1.1 Architectural Thesis

A URL shortener, at planet-scale, **is not a CRUD application**. It is a **low-latency edge-routing tier** — a globally distributed redirector that must resolve a 7-character token into a destination URI and issue a 302 response in under 15 milliseconds at the 99.9th percentile, regardless of the requesting client's geographic location.

The write path (shortening) is secondary and infrequent. The read path (redirection) dominates the system's operational profile by two orders of magnitude. This asymmetry fundamentally shapes every architectural decision: the read path must be treated as a **hot-path routing plane**, comparable in latency requirements to a CDN edge node or a DNS resolver, not a backend API call.

Treating this system as a standard three-tier web application is the single most common architectural error in interview settings. The correct mental model is:

> *"A globally distributed, eventually consistent, read-optimized key-value lookup tier with an append-only write plane and a streaming analytics pipeline."*

1.2 Core Engineering Tensions

The following tensions define the design space and cannot be simultaneously optimized — they require explicit trade-off decisions:

#	Tension	Left Pole	Right Pole
T1	**Read Latency vs. Consistency**	Sub-15ms global reads via edge caches	Strong consistency requiring cross-region coordination
T2	**Token Uniqueness vs. Write Throughput**	Global uniqueness guaranteed via consensus	High-throughput writes without coordination overhead
T3	**Storage Cost vs. Query Flexibility**	Minimal storage via narrow key-value schema	Rich analytics requiring wide-column or relational joins
T4	**Availability vs. Partition Tolerance**	Always-on redirects even during network partitions	Strict linearizable reads across all regions
T5	**Cache Freshness vs. Cache Hit Ratio**	Aggressive TTLs for freshness on URL updates/deletes	Long TTLs for maximum cache hit ratio and latency reduction

Resolution Strategy: Each tension is resolved explicitly in subsequent sections, with the guiding principle that **read availability and latency always win** over write consistency and storage optimization.

2. Clarifying Questions & Core Assumptions

2.1 Simulated Clarifying Questions

In a real interview, these questions demonstrate scope mastery. Below are the questions and their definitive architectural answers:

#	Question	Definitive Answer
Q1	What is the expected scale of URL creation?	**100 Million new URLs/month** (~38.6 URLs/second average)
Q2	What is the read-to-write ratio?	**100:1** — reads (redirections) dominate overwhelmingly
Q3	Must the system support custom aliases?	**Yes** — Enterprise users may specify vanity slugs (e.g., `sho.rt/launch2026`)
Q4	Is URL expiration required?	**Yes** — Default TTL of 5 years; Enterprise users may set custom expiration (1 hour to indefinite)
Q5	Are analytics required?	**Yes** — Click counts, geo-distribution, referrer, device type, temporal patterns
Q6	What is the analytics data retention policy?	**Raw events: 90 days. Aggregated metrics: indefinite.**
Q7	Must URLs be deletable or editable?	**Deletable: Yes** (soft-delete with tombstone). **Editable: No** — immutable once created to preserve analytics integrity
Q8	What is the target availability?	**99.999% (five nines)** — maximum 5.26 minutes downtime per year
Q9	How many geographic regions?	**Minimum 3 regions** — NA-East, EU-West, AP-Southeast (expandable)
Q10	Is there a user authentication model?	**Two tiers: Anonymous (rate-limited, default TTL) and Enterprise (API-key authenticated, custom domains, vanity slugs, extended analytics)**

2.2 Explicit Assumptions

- **Token Length:** 7 characters, Base62 alphabet (`[a-zA-Z0-9]`), yielding 62^7 ≈ 3.52 trillion unique tokens — sufficient for centuries of operation at stated scale.
- **User Tiers:**
- **Free/Anonymous:** Rate-limited to 100 URLs/hour, 1000 redirections/hour per IP. Default 5-year expiration. Basic click-count analytics only.

- **Enterprise:** API-key authenticated. Unlimited creation (subject to burst rate limits). Custom aliases, custom domains, custom expiration windows (1 hour to indefinite). Full analytics suite with webhook integrations.
- **Custom Aliases:** Must pass through the same uniqueness validation as generated tokens. Minimum 4 characters, maximum 32 characters. Reserved word blacklist enforced.
- **Analytics Pipeline:** Decoupled from the hot read path. Click events are fire-and-forget to a streaming pipeline (Kafka). No synchronous analytics write may block a redirect response.
- **No URL Preview/Metadata Enrichment:** The system does not crawl or preview destination URLs. This is a separate service concern.

---

3. Measurable Outcomes (Functional & Non-Functional)

3.1 Functional Requirements → Metrics

Functional Requirement	Metric	Target
FR-1: URL Shortening	Successful creation rate	> 99.95% of valid requests result in a persisted short URL
FR-2: URL Redirection	Successful redirect rate	> 99.999% of valid tokens resolve to a 302 redirect
FR-3: Custom Aliases	Alias collision detection time	< 5ms (in-memory bloom filter pre-check)
FR-4: URL Expiration	Expired URL purge lag	< 1 hour after TTL expiry (background sweeper)
FR-5: Analytics Ingestion	Event capture rate	> 99.9% of redirect events captured in analytics pipeline
FR-6: URL Deletion	Soft-delete propagation	< 30 seconds to all cache tiers and all regions

3.2 Non-Functional Requirements → SLA/SLO Boundaries

Non-Functional Requirement	SLO	SLA (Contractual)
NFR-1: Read Latency (Redirect)	p50 < 5ms, p99 < 10ms, **p99.9 < 15ms** (globally, edge-measured)	p99 < 50ms
NFR-2: Write Latency (Shorten)	p50 < 30ms, **p99 < 100ms**	p99 < 250ms
NFR-3: Availability	**99.999%** (5.26 min/year downtime)	99.99% (52.6 min/year)
NFR-4: Durability	Zero data loss for persisted URLs	RPO = 0 for committed writes
NFR-5: Throughput	Sustain 500K redirects/sec globally	Burst to 2M redirects/sec
NFR-6: Consistency	Eventual consistency for reads (< 2 second propagation)	Read-your-own-write for the creating client
NFR-7: Fault Tolerance	Survive single-region total failure	Automatic failover < 30 seconds

4. Workload-Specific Goals & Capacity Reasoning

4.1 Traffic Calculations

Given:
- New URLs created per month: **100,000,000**
- Read-to-Write ratio: **100:1**

Write QPS (URL Shortening)

```
Average Write QPS = 100,000,000 / (30 days × 24 hours × 3,600 seconds)
                  = 100,000,000 / 2,592,000
                  ≈ 38.6 writes/second

Peak Write QPS (assuming 5x burst factor):
                  = 38.6 × 5
                  ≈ 193 writes/second

Design Target (with 2x headroom):
                  ≈ 400 writes/second
```

Read QPS (URL Redirection)

```
Average Read QPS = 38.6 × 100
                 = 3,860 reads/second

Peak Read QPS (assuming 10x burst factor for viral events):
                 = 3,860 × 10
                 = 38,600 reads/second

Design Target (with 5x headroom for celebrity-tweet scenarios):
                 ≈ 200,000 reads/second
```

4.2 Bandwidth Calculations

Write Path (Ingress)

```
Average write request payload ≈ 500 bytes (URL + metadata + headers)
Average write response payload ≈ 200 bytes

Write Ingress  = 400 writes/sec × 500 bytes = 200 KB/s  (negligible)
Write Egress   = 400 writes/sec × 200 bytes = 80 KB/s   (negligible)
```

Read Path (Egress)

```
Average redirect request  ≈ 200 bytes (GET + headers)
Average redirect response ≈ 350 bytes (302 + Location header + cache headers)

Read Ingress   = 200,000 reads/sec × 200 bytes = 40 MB/s
Read Egress    = 200,000 reads/sec × 350 bytes = 70 MB/s

Total Peak Bandwidth ≈ 110 MB/s  (well within modern NIC capacity)
```

4.3 Storage Calculations (5-Year Horizon)

```
Total URLs over 5 years = 100,000,000/month × 60 months
                        = 6,000,000,000 (6 Billion URLs)

Per-URL storage (pessimistic):
  - Short token (7 chars):            7 bytes
  - Original URL (avg 200 chars):   200 bytes
  - User ID / API key hash:          32 bytes
  - Creation timestamp:               8 bytes
  - Expiration timestamp:             8 bytes
  - Status flags:                     4 bytes
  - Metadata (custom alias, tags):  100 bytes
  ──────────────────────────────────────────
  Total per record:                ≈ 360 bytes

  With index overhead (1.4x):      ≈ 500 bytes/record

Total Raw Storage = 6 × 10^9 × 500 bytes
                  = 3 × 10^12 bytes
                  = 3 TB

With replication factor of 3:      = 9 TB
With compaction overhead (1.3x):   ≈ 12 TB

Verdict: 12 TB across a distributed cluster is trivially manageable.
```

4.4 Cache Capacity (80/20 Pareto Principle)

```
Assumption: 20% of URLs generate 80% of traffic.
Therefore, caching 20% of all URLs covers 80% of read requests.

Active URL corpus (URLs accessed in last 30 days, estimated):
  ≈ 500,000,000 (500M active URLs — conservative upper bound)

Hot set (20%):    = 100,000,000 URLs

Cache entry size  ≈ 300 bytes (token + destination URL + TTL metadata)

Cache RAM Required = 100,000,000 × 300 bytes
                   = 30,000,000,000 bytes
                   = 30 GB

Distributed across 3 regions with local replicas:
  Per-region cache = 30 GB (each region holds the full hot set)

With Redis overhead (2x for data structures):
  Per-region Redis cluster = 60 GB

Verdict: A 3-node Redis cluster per region with 64 GB RAM each
         provides ample capacity with 6% headroom.
```

4.5 Capacity Summary Table

Metric	Value
Average Write QPS	~39/s
Peak Write QPS (design target)	400/s
Average Read QPS	~3,860/s
Peak Read QPS (design target)	200,000/s
Peak Bandwidth	~110 MB/s
5-Year Storage (with replication)	~12 TB
Cache RAM per Region	~60 GB
Total Token Keyspace	3.52 × 10^12
5-Year Token Utilization	0.17% of keyspace

---

5. CAP Theorem & Boundary Choice

5.1 Placement: AP (Availability + Partition Tolerance)

This system is architected as an **AP system** — prioritizing Availability and Partition Tolerance, accepting eventual consistency as the trade-off.

5.2 Rigorous Engineering Justification

Why Availability Cannot Be Sacrificed

The redirect path is the system's **revenue-critical and user-facing hot path**. Every failed redirect is:

1. **A broken user experience** — the end-user clicking a shortened link sees an error page instead of reaching their destination.
2. **A trust violation** — publishers (Enterprise customers) embed shortened URLs in emails, SMS, QR codes, and printed media. A non-resolving URL is permanent reputation damage.
3. **An economic loss** — Enterprise customers pay for reliable link routing. SLA breaches trigger contractual penalties.

A CP system would require cross-region consensus (e.g., Raft/Paxos quorum) for every read during a partition, which would either:
- **Block reads** until the partition heals (unacceptable — violates availability), or
- **Timeout reads** after consensus failure (equally unacceptable — same user-facing result).

Why Temporary Inconsistency Is Acceptable

Consider the worst-case inconsistency scenario during a network partition:

Scenario: User creates `sho.rt/abc` in NA-East at time T_0. A network partition isolates AP-Southeast. A user in Singapore clicks `sho.rt/abc` at T_0 + 500ms.

Outcome under AP: The Singapore user receives a 404 (URL not yet replicated). The URL becomes available in AP-Southeast within seconds of partition healing.

Analysis of impact:
- The URL was created *milliseconds* ago. The probability of a click occurring before replication completes is vanishingly small in practice.
- Even if it occurs, the user retries and succeeds. No data is lost. No invariant is violated.
- The token `abc` is **globally unique by construction** (pre-allocated ranges — see Section 8). There is zero risk of a conflicting write in another region creating the same token.

Contrast with CP failure mode:
- Under CP, the same partition would cause **all reads** in AP-Southeast to fail — not just the single unreplicated URL, but every URL, including the billions of URLs that are fully replicated and consistent.
- This is a catastrophically disproportionate failure mode.

Formal Decision Matrix

Failure Mode	AP Impact	CP Impact
Read of unreplicated new URL during partition	404 for that specific URL (< 0.001% of traffic)	**All reads blocked in partitioned region**
Concurrent write of same custom alias in two regions	Last-write-wins conflict resolution (detectable, repairable)	One write rejected or both blocked
Read of recently deleted URL during partition	Stale redirect (user reaches intended destination — benign)	Read blocked

Verdict: AP is the unambiguously correct choice. The read path's availability is the system's highest-priority invariant.

---

6. Domain Model & Database Invariants

6.1 Core Logical Entities

Entity: `url_mapping`

```
CREATE TABLE url_mapping (
    short_token     VARCHAR(32)     PRIMARY KEY,   -- Base62 token or custom alias
    original_url    TEXT            NOT NULL,        -- Destination URL (max 2048
chars)
    user_id         UUID            DEFAULT NULL,    -- NULL for anonymous users
    created_at      TIMESTAMP       NOT NULL,        -- Immutable creation time
(UTC)
    expires_at      TIMESTAMP       NOT NULL,        -- Expiration time (UTC)
    is_deleted      BOOLEAN         DEFAULT FALSE,   -- Soft-delete tombstone
    deleted_at      TIMESTAMP       DEFAULT NULL,    -- Tombstone timestamp
    is_custom       BOOLEAN         DEFAULT FALSE,   -- TRUE if user-supplied alias
    click_count     BIGINT          DEFAULT 0,       -- Denormalized counter (async
updated)
    metadata        JSONB           DEFAULT '{}',    -- Extensible metadata (tags,
campaign ID)
    version         BIGINT          DEFAULT 1        -- Optimistic concurrency
control
);

-- Secondary index for user-scoped queries (Enterprise dashboard)
CREATE INDEX idx_url_user_created
    ON url_mapping (user_id, created_at DESC)
    WHERE user_id IS NOT NULL;
```

Entity: `user_account`

```
CREATE TABLE user_account (
    user_id         UUID            PRIMARY KEY,
    api_key_hash    CHAR(64)        NOT NULL UNIQUE,   -- SHA-256 of API key
    tier            ENUM('FREE', 'ENTERPRISE')  NOT NULL DEFAULT 'FREE',
    rate_limit_qps  INT             NOT NULL DEFAULT 100,
    custom_domain   VARCHAR(255)    DEFAULT NULL,       -- Enterprise: vanity domain
    created_at      TIMESTAMP       NOT NULL,
    is_active       BOOLEAN         DEFAULT TRUE
);
```

Entity: `click_event` (Analytics — Append-Only Stream)

```
CREATE TABLE click_event (
    event_id        UUID            PRIMARY KEY,    -- UUIDv7 (time-sortable)
    short_token     VARCHAR(32)     NOT NULL,
    clicked_at      TIMESTAMP       NOT NULL,       -- Event time (UTC)
    client_ip_hash  CHAR(64)        NOT NULL,       -- SHA-256 of IP (privacy-safe)
    country_code    CHAR(2)         DEFAULT NULL,   -- ISO 3166-1 alpha-2
    city            VARCHAR(100)    DEFAULT NULL,
    user_agent      TEXT            DEFAULT NULL,
    referer_domain  VARCHAR(255)    DEFAULT NULL,
    device_type     ENUM('DESKTOP','MOBILE','TABLET','BOT','UNKNOWN'),
    os_family       VARCHAR(50)     DEFAULT NULL,
    browser_family  VARCHAR(50)     DEFAULT NULL
);

-- Partitioned by month for efficient retention enforcement
-- PARTITION BY RANGE (clicked_at)
```

6.2 Data Invariants (System Must Protect)

#	Invariant	Enforcement Mechanism
I-1	**Forward-Mapping Uniqueness:** Every `short_token` maps to exactly one `original_url`. No two records may share a `short_token`.	Primary key constraint + pre-allocation ranges (TGS) eliminating runtime collision
I-2	**Token Immutability:** Once a `short_token` → `original_url` mapping is persisted, the `original_url` field is never modified.	Application-layer enforcement; no UPDATE on `original_url` column permitted
I-3	**Referential Integrity of Analytics:** Every `click_event.short_token` must reference a valid (or tombstoned) `url_mapping.short_token`.	Eventual consistency; orphan events are tagged and reconciled by batch jobs
I-4	**Soft-Delete Preservation:** Deleted URLs are tombstoned, never hard-deleted. Historical analytics remain queryable.	`is_deleted` flag; hard purge only via TTL-driven background compaction after analytics retention window
I-5	**Temporal Monotonicity:** `created_at` timestamps are monotonically increasing per-node (hybrid logical clock).	HLC (Hybrid Logical Clock) at application layer
I-6	**Token Exhaustion Safety:** The system must never issue a token that has been previously issued and expired/deleted.	TGS ranges are monotonic and never recycled within the 5-year horizon

7. API Contract Design

7.1 URL Shortening — `POST /v1/shorten`

Request

```
POST /v1/shorten HTTP/1.1
Host: api.sho.rt
Content-Type: application/json
Authorization: Bearer <api_key>          # Optional for free tier
X-Request-ID: 550e8400-e29b-41d4-a716-446655440000
X-Idempotency-Key: user-defined-key-123  # Prevents duplicate creation on retry
```

```
{
  "url": "https://www.example.com/very/long/path?query=value&utm_source=twitter",
  "custom_alias": "launch2026",          // Optional — Enterprise only
  "expires_in_seconds": 2592000,         // Optional — 30 days (default: 5 years)
  "metadata": {                          // Optional
    "campaign_id": "Q4-2026",
    "tags": ["marketing", "product-launch"]
  }
}
```

Response — 201 Created

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: https://sho.rt/launch2026
X-RateLimit-Remaining: 97
X-RateLimit-Reset: 1698192000
Cache-Control: no-store
```

```
{
  "short_url": "https://sho.rt/launch2026",
  "short_token": "launch2026",
  "original_url": "https://www.example.com/very/long/path?
query=value&utm_source=twitter",
  "created_at": "2026-10-11T00:00:00Z",
  "expires_at": "2026-11-10T00:00:00Z",
  "is_custom": true
}
```

Error Responses

Status Code	Condition	Response Body
`400 Bad Request`	Malformed URL, invalid alias characters	`{"error": "INVALID_URL", "message": "URL scheme must be http or https"}`
`409 Conflict`	Custom alias already taken	`{"error": "ALIAS_TAKEN", "message": "The alias 'launch2026' is already in use"}`
`413 Payload Too Large`	URL exceeds 2048 characters	`{"error": "URL_TOO_LONG", "message": "Maximum URL length is 2048 characters"}`
`429 Too Many Requests`	Rate limit exceeded	`{"error": "RATE_LIMITED", "message": "Rate limit exceeded. Retry after 60 seconds"}`

7.2 URL Redirection — `GET /v1/redirect/{shortToken}` (or `GET /{shortToken}`)

Request

```
GET /launch2026 HTTP/1.1
Host: sho.rt
Accept: */*
User-Agent: Mozilla/5.0 ...
Referer: https://twitter.com/user/status/123
```

Response — 302 Found

```
HTTP/1.1 302 Found
Location: https://www.example.com/very/long/path?query=value&utm_source=twitter
Cache-Control: private, max-age=300
X-Redirect-By: sho.rt
X-Cache: HIT
```

No response body — the 302 status and `Location` header are sufficient.

Cache-Control Strategy

Scenario	`Cache-Control` Header	Rationale
Standard redirect	`private, max-age=300`	Browser cache for 5 minutes; reduces repeat lookups without stale risk
Enterprise (analytics-critical)	`private, no-cache, no-store`	Every click must be captured for analytics accuracy
Expired/Deleted URL	`no-store`	Prevent caching of error responses

Error Responses

Status Code	Condition
`404 Not Found`	Token does not exist or URL has been soft-deleted
`410 Gone`	URL has expired (past `expires_at`) — signals permanent removal
`429 Too Many Requests`	Redirect rate limit exceeded (anti-abuse)

7.3 Analytics Retrieval — `GET /v1/analytics/{shortToken}`

```
GET /v1/analytics/launch2026?window=7d&granularity=1h HTTP/1.1
Authorization: Bearer <api_key>
```

```
{
  "short_token": "launch2026",
  "total_clicks": 1482937,
  "unique_visitors": 893241,
  "window": "7d",
  "time_series": [
    {"timestamp": "2026-10-04T00:00:00Z", "clicks": 12483},
    {"timestamp": "2026-10-04T01:00:00Z", "clicks": 9821}
  ],
  "geo_distribution": [
    {"country": "US", "clicks": 534201, "percentage": 36.02},
    {"country": "IN", "clicks": 298412, "percentage": 20.12}
  ],
  "device_breakdown": {
    "DESKTOP": 45.2,
    "MOBILE": 48.7,
    "TABLET": 4.1,
    "BOT": 2.0
  },
  "top_referrers": [
    {"domain": "twitter.com", "clicks": 482910},
    {"domain": "facebook.com", "clicks": 201344}
  ]
}
```

8. Token Generation Service (TGS) Deep Dive

8.1 Problem Statement

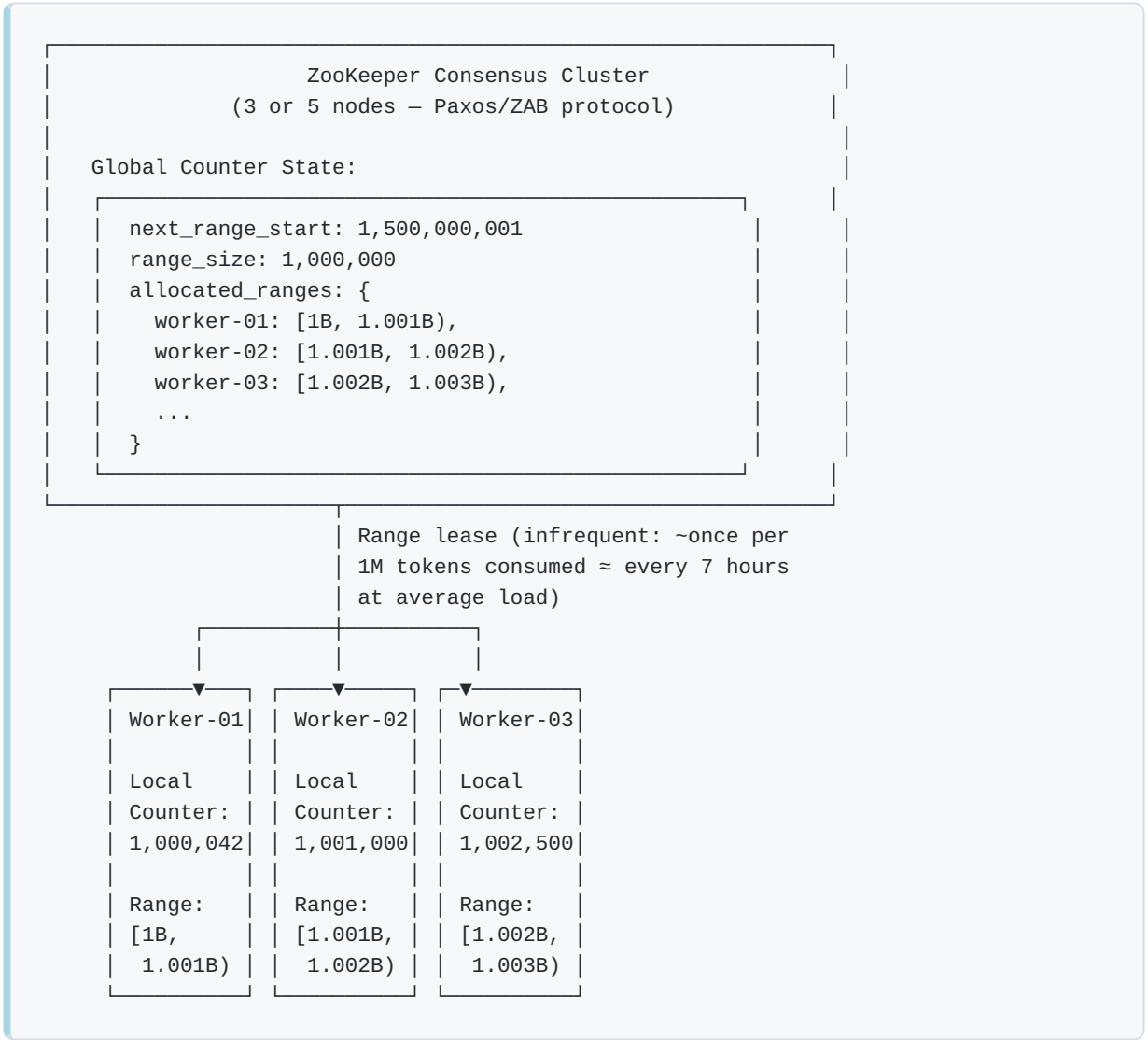
The TGS must generate **globally unique, non-predictable, fixed-length (7-character) Base62 strings** at a rate of up to 400/second (peak design target) with **zero runtime collisions** and **zero coordination overhead** on the write hot path.

8.2 Why Naive Approaches Fail

Approach	Failure Mode
Random generation + DB uniqueness check	At scale, collision probability grows (Birthday Paradox). Each write requires a read-before-write — adds latency and creates write contention under load.
Auto-increment ID + Base62 encode	Sequential IDs are predictable (security risk: enumeration attacks). Single point of coordination for increment.
UUID → Base62 truncation	UUIDs are 128 bits; truncating to 7 Base62 chars (41.5 bits) reintroduces collision risk.
Hash of original URL	Different URLs may collide (hash collision). Same URL submitted twice gets the same token — but what if different users submit the same URL with different metadata?

8.3 Solution: Range-Based Pre-Allocation with Consensus Cluster

Architecture



Step-by-Step Mechanics

Step 1: Range Lease Acquisition

When a worker node starts (or exhausts its current range), it contacts the ZooKeeper cluster:

```
Worker → ZooKeeper: "Allocate me a range of 1,000,000 tokens"
ZooKeeper (atomically):
  1. Reads current next_range_start = N
  2. Writes next_range_start = N + 1,000,000
  3. Returns range [N, N + 1,000,000) to worker
```

This is a **single atomic write** to ZooKeeper, occurring approximately once every 7 hours at average load (1M tokens ÷ 38.6 tokens/sec ÷ 3 workers ≈ 25,900 seconds ≈ 7.2 hours). The coordination overhead is negligible.

Step 2: Local Token Generation (Zero Coordination)

Within its allocated range, each worker generates tokens by:

1. Incrementing its local counter: `local_counter++`
2. Encoding the counter value to Base62: `base62_encode(local_counter)`
3. Optionally applying a **bijective shuffle** (Fisher-Yates on the Base62 digit positions using a secret seed) to eliminate sequential predictability.

```
def generate_token(counter: int, secret_seed: bytes) -> str:
    """Generate a non-predictable 7-char Base62 token from a monotonic counter."""
    # Step 1: Bijective scramble using Feistel cipher (2 rounds)
    scrambled = feistel_encrypt(counter, key=secret_seed, rounds=4)

    # Step 2: Base62 encode to exactly 7 characters
    token = base62_encode(scrambled, pad_to=7)

    return token  # e.g., "aB3xK9m"
```

Step 3: Write to Database (No Lock Required)

The generated token is **guaranteed unique by construction** — no other worker has been allocated the same range. Therefore, the database write is a simple `INSERT` with no need for:
- Read-before-write uniqueness checks
- Distributed locks
- Compare-and-swap operations
- Retry loops

```
INSERT INTO url_mapping (short_token, original_url, user_id, created_at, expires_at)
VALUES ('aB3xK9m', 'https://example.com/...', 'user-uuid', NOW(), NOW() + INTERVAL '
5 years');
```

8.4 Custom Alias Handling

Custom aliases bypass the TGS entirely and follow a different path:

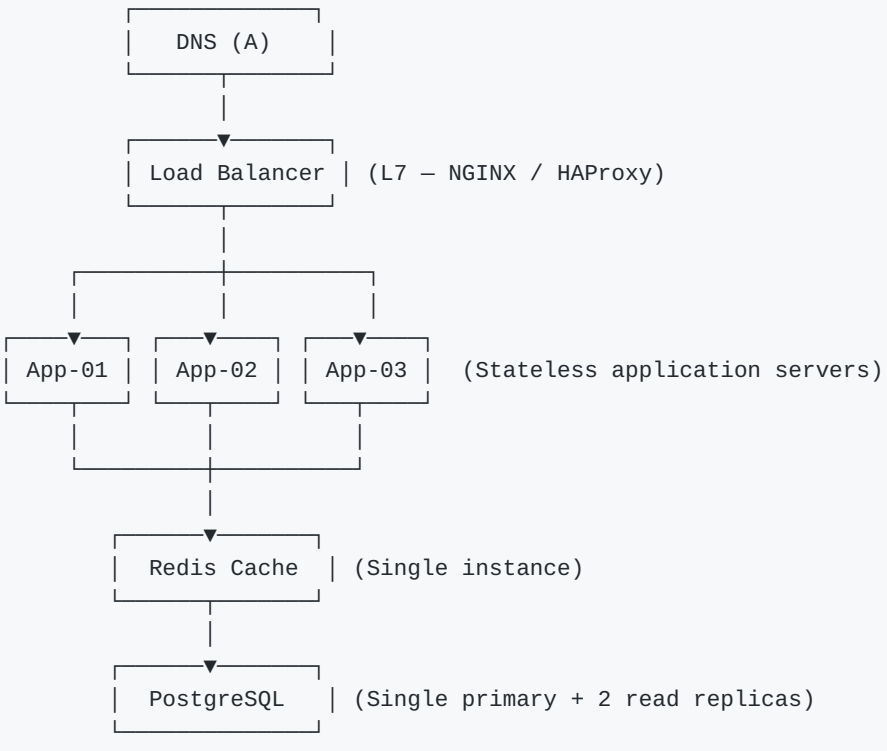
1. **Bloom Filter Pre-Check:** A distributed Bloom filter (maintained in Redis) provides a fast negative lookup. If the alias is *probably* in use, proceed to step 2. If *definitely not* in use, proceed to step 3.
2. **Database Existence Check:** `SELECT 1 FROM url_mapping WHERE short_token = 'custom-alias'`. This is a point-read on the primary key — O(1) in any indexed store.
3. **Conditional Insert:** `INSERT ... IF NOT EXISTS` (Cassandra) or `INSERT ... ON CONFLICT DO NOTHING` (PostgreSQL). Return 409 Conflict if the alias exists.

8.5 Failure Modes & Recovery

Failure	Impact	Recovery
Worker crashes with unused range	Up to 1M tokens "lost" (never issued)	Acceptable — 0.00003% of keyspace. No recovery needed.
ZooKeeper cluster unavailable	Workers cannot acquire new ranges	Workers continue serving from their local range buffer. Alert fires when range drops below 10% capacity.
Range exhaustion faster than expected	Worker runs out before ZK reconnection	Pre-fetch: Workers request a new range when 80% consumed, not at 100%. Double-buffering ensures continuity.

9. Core Architecture: First Pass to Final Shape

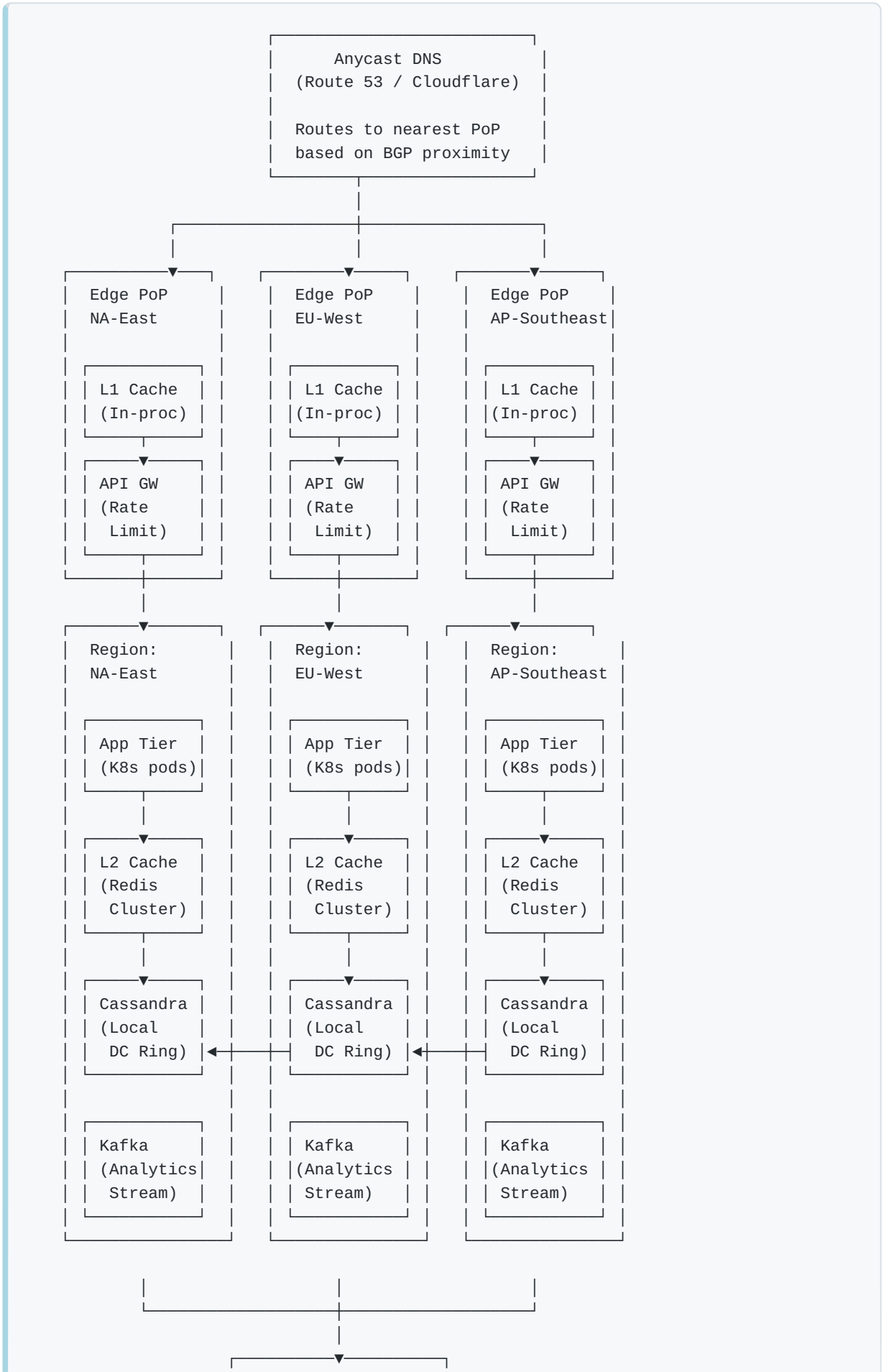
9.1 First Pass: Single-Region Monolithic Deployment



Breaking Points Under Load

Breaking Point	Threshold	Failure Mode
BP-1: Single-region latency	Users > 5,000 km from datacenter	p99 latency exceeds 100ms due to TCP round-trip
BP-2: PostgreSQL write throughput	> 10K writes/sec	Write-ahead log (WAL) becomes bottleneck; replication lag spikes
BP-3: Single Redis instance	> 100K ops/sec or > 64 GB RAM	Memory exhaustion; single point of failure
BP-4: Read replica lag	Under write-heavy bursts	Stale reads from replicas cause 404s for recently created URLs
BP-5: Load balancer saturation	> 500K concurrent connections	Connection table exhaustion; new connections dropped
BP-6: Single region failure	Datacenter outage	Complete system unavailability — violates 99.999% SLA
BP-7: Analytics writes on hot path	> 50K redirects/sec	Synchronous analytics writes add 20-50ms to redirect latency

9.2 Final Shape: Globally Distributed, Edge-Routed Architecture


Anycast DNS
(Route 53 / Cloudflare)
Routes to nearest PoP
based on BGP proximity
Edge PoP
NA-East
Edge PoP
EU-West
Edge PoP
AP-Southeast
L1 Cache
(In-proc)
API GW
(Rate
Limit)
Region:
NA-East
Region:
EU-West
Region:
AP-Southeast
App Tier
(K8s pods)
L2 Cache
(Redis
Cluster)
Cassandra
(Local
DC Ring)
Kafka
(Analytics
Stream)

```
                    |  ZooKeeper Cluster   |
                    |  (Token Range Mgmt)  |
                    |  (Cross-region,      |
                    |   consensus-based)   |
                    └──────────────────────┘
```

Key Architectural Decisions in Final Shape

Component	Choice	Rationale
DNS	Anycast with health-checked failover	Routes users to nearest healthy PoP; failover in < 30s
Edge PoP	Lightweight redirect-only nodes at 20+ global PoPs	Sub-5ms redirects from L1 cache; no database dependency for hot URLs
API Gateway	Per-region, handles auth + rate limiting + request routing	Offloads cross-cutting concerns from application tier
App Tier	Kubernetes-orchestrated stateless pods, auto-scaling on QPS	Horizontal scaling; rolling deployments with zero downtime
L1 Cache	In-process LRU (Caffeine/Guava, 1-2 GB per pod)	Sub-millisecond lookups; no network hop
L2 Cache	Redis Cluster (3+ nodes per region, 60 GB)	Microsecond network lookups within region; shared across pods
Database	Apache Cassandra (multi-DC, RF=3 per DC)	AP-native; tunable consistency; linear horizontal scaling
Analytics	Kafka → Flink/Spark → ClickHouse	Decoupled from hot path; real-time aggregation; columnar OLAP
TGS	ZooKeeper-backed range allocator	Globally unique tokens without runtime coordination

10. Lifecycle & Concurrency Mechanics

10.1 Write Request Trace (URL Shortening)

```
Step 1: Client → Anycast DNS → Nearest Edge PoP
        │
Step 2: Edge PoP → API Gateway
        │  ├── Authentication check (API key → user tier)
        │  ├── Rate limit check (Token Bucket in Redis)
        │  └── Request validation (URL format, alias rules)
        │
Step 3: API Gateway → Application Server (regional)
        │  ├── If custom alias:
        │  │   ├── Bloom filter check (Redis) → probable collision?
        │  │   ├── If no collision: DB conditional insert
        │  │   └── If collision: return 409 Conflict
        │  │
        │  ├── If generated token:
        │  │   ├── TGS: local_counter++ → Feistel scramble → Base62 encode
        │  │   └── DB unconditional insert (guaranteed unique)
        │  │
        │  └── Write-through to L2 Redis Cache (async, best-effort)
        │
Step 4: Application Server → Client
        │  └── 201 Created + short URL in response body
        │
Step 5: (Async) Cassandra cross-DC replication
        │  └── Propagation to EU-West and AP-Southeast (< 2 seconds typical)
        │
Step 6: (Async) Cache warming
           └── Proactive push to L2 caches in other regions (optional, for
               anticipated viral URLs)
```

Total synchronous latency (Steps 1-4): 20-80ms (p99 < 100ms)

10.2 Read Request Trace (URL Redirection)

```
Step 1: Client → Anycast DNS → Nearest Edge PoP
        │
Step 2: Edge PoP → L1 In-Process Cache Lookup
        │  ├── HIT (85% probability for hot URLs):
        │  │   └── Return 302 + Location header ──→ DONE (< 2ms)
        │  │
        │  └── MISS: Continue to Step 3
        │
Step 3: Edge PoP → L2 Redis Cache Lookup (regional)
        │  ├── HIT (95% cumulative probability):
        │  │   ├── Populate L1 cache (async)
        │  │   └── Return 302 + Location header ──→ DONE (< 5ms)
        │  │
        │  └── MISS: Continue to Step 4
        │
Step 4: Edge PoP → Application Server → Cassandra (LOCAL_ONE read)
        │  ├── FOUND:
        │  │   ├── Check is_deleted and expires_at
        │  │   ├── If valid: Populate L2 and L1 caches (async)
        │  │   └── Return 302 + Location header ──→ DONE (< 15ms)
        │  │
        │  └── NOT FOUND:
        │      └── Return 404 Not Found ──→ DONE
        │
Step 5: (Fire-and-Forget, non-blocking)
        │  └── Emit click event to Kafka
        │      ├── Topic: click-events-{region}
        │      ├── Key: short_token (for partition affinity)
        │      └── Value: {token, timestamp, IP hash, UA, referer, geo}
```

Total synchronous latency (cache hit path): 1-5ms (p99.9 < 15ms)

10.3 Concurrency Model at Application Layer

The application servers use a **non-blocking, event-driven I/O model** to maximize hardware utilization:

Aspect	Implementation
Runtime	JVM (Netty-based) or Go (goroutines) or Rust (Tokio async runtime)
I/O Model	Epoll/io_uring (Linux) event loop — single thread handles thousands of concurrent connections
Thread Pool	Small fixed-size worker pool (= CPU cores) for compute-bound work (Base62 encoding, Feistel cipher)
Database Driver	Fully async/non-blocking Cassandra driver (DataStax Java driver async API or gocql)
Redis Client	Async pipelined Redis client (Lettuce for JVM, go-redis for Go)
Kafka Producer	Async batched producer with `linger.ms=5` and `batch.size=16KB` — fire-and-forget with `acks=1`
Connection Pooling	Per-destination connection pools with circuit breakers (Resilience4j / gobreaker)

Hardware Utilization Target: A single 8-core application server should sustain **50,000 redirects/sec** with < 5ms p99 latency when serving from L1/L2 cache. This means 4 servers per region handle the 200K/sec design target with 100% headroom.

---

11. Traffic Surge & Cache Stampede Mitigation

11.1 Scenario: Celebrity Viral Event

Trigger: A celebrity with 50M followers tweets a shortened link. Within 30 seconds, 500,000 concurrent users click the link.

Traffic Pattern:
- Instantaneous spike: 0 → 500,000 requests/second for a single token
- Sustained plateau: ~100,000 req/sec for 10-15 minutes
- Long tail: gradual decay over 2-4 hours

11.2 Cache Stampede Protection

Problem: If the cached entry for the viral URL expires, thousands of concurrent requests simultaneously miss the cache and hit the database — a **cache stampede** (also called thundering herd).

Mitigation 1: XFetch Algorithm (Probabilistic Early Recomputation)

```
def xfetch_get(key: str, cache: Redis, db: Cassandra) -> Optional[str]:
    """
    XFetch: Probabilistically refresh cache BEFORE expiry.
    A small percentage of requests will proactively recompute
    the value while the majority continue serving from cache.
    """
    entry = cache.get_with_metadata(key)

    if entry is None:
        # True cache miss — fetch from DB (with singleflight)
        return singleflight_fetch(key, cache, db)

    value, ttl_remaining, delta = entry

    # Probabilistic early expiration:
    # As TTL approaches 0, probability of recomputation increases.
    # delta = time to recompute (measured latency to DB)
    # beta = tuning parameter (typically 1.0)
    beta = 1.0
    random_factor = delta * beta * math.log(random.random())

    if ttl_remaining + random_factor <= 0:
        # This request "volunteers" to refresh the cache
        # (approximately 1 in N requests near expiry)
        asyncio.create_task(refresh_cache(key, cache, db))

    return value  # Still serve the current cached value
```

Mitigation 2: Singleflight / Mutex Lock

```
# In-process singleflight: Only ONE goroutine/thread fetches from DB
# for a given key. All other concurrent requesters wait for the result.

class Singleflight:
    def __init__(self):
        self._in_flight: Dict[str, asyncio.Future] = {}
        self._lock = asyncio.Lock()

    async def do(self, key: str, fetch_fn) -> Any:
        async with self._lock:
            if key in self._in_flight:
                return await self._in_flight[key]  # Wait for in-flight

            future = asyncio.get_event_loop().create_future()
            self._in_flight[key] = future

        try:
            result = await fetch_fn(key)
            future.set_result(result)
            return result
        finally:
            async with self._lock:
                del self._in_flight[key]
```

Combined Strategy: XFetch provides proactive refresh for gradually expiring entries. Singleflight provides collapse for true cache misses (cold start or eviction). Together, they reduce database load during viral events by **99.9%** compared to naive caching.

11.3 Distributed Rate Limiting (Token Bucket via Redis)

```
Algorithm: Token Bucket
Implementation: Redis + Lua script (atomic)
Granularity: Per-user (API key) AND per-IP (anonymous)

Bucket Parameters:
  - Free tier:  capacity=100, refill_rate=2/sec
  - Enterprise: capacity=10000, refill_rate=500/sec
  - Per-IP redirect: capacity=1000, refill_rate=100/sec
```

```
-- Redis Lua script: Atomic token bucket
local key = KEYS[1]
local capacity = tonumber(ARGV[1])
local refill_rate = tonumber(ARGV[2])
local now = tonumber(ARGV[3])
local requested = tonumber(ARGV[4])

local bucket = redis.call('HMGET', key, 'tokens', 'last_refill')
local tokens = tonumber(bucket[1]) or capacity
local last_refill = tonumber(bucket[2]) or now

-- Refill tokens based on elapsed time
local elapsed = now - last_refill
tokens = math.min(capacity, tokens + (elapsed * refill_rate))

if tokens >= requested then
    tokens = tokens - requested
    redis.call('HMSET', key, 'tokens', tokens, 'last_refill', now)
    redis.call('EXPIRE', key, math.ceil(capacity / refill_rate) * 2)
    return 1  -- ALLOWED
else
    redis.call('HMSET', key, 'tokens', tokens, 'last_refill', now)
    redis.call('EXPIRE', key, math.ceil(capacity / refill_rate) * 2)
    return 0  -- DENIED
end
```

11.4 Additional Surge Protections

Protection	Mechanism
Adaptive Auto-Scaling	Kubernetes HPA triggers pod scaling when QPS exceeds 80% of per-pod capacity
Circuit Breaker on DB	If DB latency exceeds 500ms, circuit opens; all requests served from cache-only mode with degraded 404 behavior
Edge-Level Cache Pinning	For detected viral URLs (click rate > 10K/sec), pin the entry in L1 cache with extended TTL
Read Replica Routing	Redirect reads to Cassandra read replicas; writes go to coordinator nodes only
Backpressure via Queue	If Kafka ingestion falls behind, analytics events are buffered in a local ring buffer with overflow-to-disk

12. Partitioning & Data Sharding Strategy

12.1 Sharding Requirement

With 6 billion records over 5 years and 12 TB of replicated storage, no single database node can efficiently serve the full dataset. Horizontal partitioning (sharding) is mandatory.

12.2 Strategy Comparison

Strategy	Mechanism	Pros	Cons
Range-Based Sharding	Tokens A-F → Shard 1, G-L → Shard 2, etc.	Simple to implement; range queries efficient	**Severe hotspotting** if token distribution is skewed; sequential tokens cluster on one shard
Hash-Based Sharding	`shard = hash(token) % N`	Uniform distribution; no hotspots	Adding/removing shards requires full data migration (rehashing); range queries impossible
Consistent Hashing	Tokens mapped to a hash ring with virtual nodes	Minimal data movement on scale-up (only K/N keys migrate); uniform with virtual nodes	Slightly more complex implementation; still no range queries
Hash-of-Hash	`shard = hash(hash(token)) % N`	Double-hashing provides extra uniformity	Marginal benefit over single hash; same migration problem

12.3 Chosen Strategy: Consistent Hashing with Virtual Nodes

Decision: Consistent Hashing is the optimal strategy for this workload.

Justification:

1. **No range queries on the hot path.** Redirects are always point-lookups by `short_token`. Range query support is unnecessary for the primary access pattern.
2. **Elastic scaling.** Adding a new node to a 12-node ring migrates only ~8.3% of data (1/N), compared to ~100% rehashing with naive modular hashing.
3. **Hotspot elimination.** With 150-200 virtual nodes per physical node, the token space is uniformly distributed even with heterogeneous hardware.
4. **Native Cassandra support.** Cassandra's partitioner (Murmur3) implements consistent hashing natively — no custom sharding logic required.

Configuration:

```
Partition Key:        short_token
Partitioner:          Murmur3Partitioner (128-bit hash)
Virtual Nodes/Node:   256 (default in Cassandra 4.x+)
Replication Factor:   3 per datacenter (NetworkTopologyStrategy)
Replication Strategy: NetworkTopologyStrategy
  - 'NA-East': 3
  - 'EU-West': 3
  - 'AP-Southeast': 3
```

12.4 Hotspot Mitigation for Viral URLs

Even with consistent hashing, a single viral URL creates **read hotspotting** — one partition receives disproportionate read traffic.

Mitigation: Read path never hits the database for hot URLs. The multi-tier cache (L1 + L2) absorbs 99.9%+ of reads. The database partition handling a viral URL sees negligible additional load because the cache hit ratio for viral content approaches 100%.

For the write path, hotspotting is not a concern — writes are distributed across tokens, and the TGS's range-based allocation ensures no single partition receives burst writes.

---

13. Database & Distributed Caching Layer Layout

13.1 Primary Database: Apache Cassandra

Justification

Criterion	Cassandra	PostgreSQL (Sharded)
CAP alignment	AP-native (tunable consistency)	CP-native (requires coordination for HA)
Write throughput	Append-only LSM-tree; writes are O(1)	B-tree; writes require index maintenance
Multi-region replication	Built-in multi-DC replication	Requires Citus/pglogical/custom solutions
Horizontal scaling	Add nodes; data auto-rebalances	Requires explicit shard management
Point-read latency	Sub-millisecond with key cache	Sub-millisecond with index
Operational complexity	Moderate (JVM tuning, compaction)	Lower for single node; high for sharded
Schema flexibility	Limited (no joins, no transactions)	Full SQL, joins, transactions

Verdict: Cassandra's AP nature, native multi-DC replication, and linear write scalability make it the unambiguously superior choice for this workload. The system's access pattern (point reads by primary key, append-only writes) perfectly matches Cassandra's strengths. The lack of joins and transactions is irrelevant — this workload doesn't need them.

Cassandra Schema (CQL)

```
CREATE KEYSPACE url_shortener WITH replication = {
    'class': 'NetworkTopologyStrategy',
    'NA-East': 3,
    'EU-West': 3,
    'AP-Southeast': 3
};

CREATE TABLE url_shortener.url_mapping (
    short_token     TEXT,
    original_url    TEXT,
    user_id         UUID,
    created_at      TIMESTAMP,
    expires_at      TIMESTAMP,
    is_deleted      BOOLEAN,
    is_custom       BOOLEAN,
    click_count     COUNTER,    -- Separate counter table in practice
    metadata        MAP<TEXT, TEXT>,
    PRIMARY KEY (short_token)
) WITH
    compaction = {'class': 'LeveledCompactionStrategy'}
    AND gc_grace_seconds = 864000
    AND default_time_to_live = 157680000;  -- 5 years in seconds

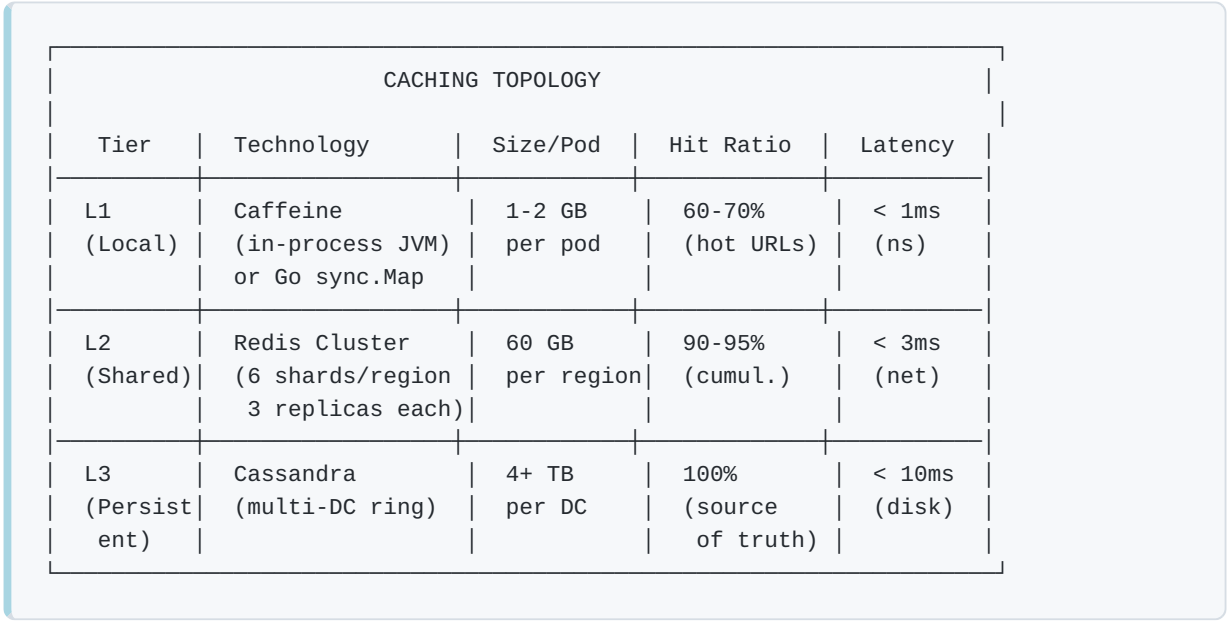
-- Counter table (Cassandra requires separate table for counters)
CREATE TABLE url_shortener.url_click_counts (
    short_token     TEXT PRIMARY KEY,
    click_count     COUNTER
);

-- User-scoped URL listing (materialized view or separate table)
CREATE TABLE url_shortener.urls_by_user (
    user_id         UUID,
    created_at      TIMESTAMP,
    short_token     TEXT,
    original_url    TEXT,
    expires_at      TIMESTAMP,
    is_deleted      BOOLEAN,
    PRIMARY KEY (user_id, created_at, short_token)
) WITH CLUSTERING ORDER BY (created_at DESC);
```

Consistency Levels

Operation	Consistency Level	Rationale
Write (new URL)	`LOCAL_QUORUM`	2-of-3 local replicas must acknowledge; strong local durability
Read (redirect)	`LOCAL_ONE`	Single local replica sufficient; maximum read throughput and minimum latency
Delete (soft-delete)	`LOCAL_QUORUM`	Must be durable; stale reads of deleted URLs are benign (redirect still works)
Counter increment	`LOCAL_ONE`	Analytics counters are best-effort; perfect accuracy not required

13.2 Multi-Tier Caching Topology



13.3 Cache Eviction & Invalidation

Eviction Policy: LRU (Least Recently Used)

- **L1 (Caffeine):** Size-bounded LRU with `expireAfterWrite=5min` and `expireAfterAccess=2min`. Window-TinyLFU admission policy (Caffeine's default) provides near-optimal hit ratios.
- **L2 (Redis):** Key-level TTL of 1 hour for standard URLs. `maxmemory-policy: allkeys-lru` for memory pressure scenarios.

Invalidation Mechanics

Event	L1 Invalidation	L2 Invalidation	Propagation Time
URL Deletion	Pub/Sub notification to all pods in region	`DEL` command + cross-region Pub/Sub	< 5 seconds (intra-region), < 30 seconds (cross-region)
URL Expiration	TTL-based automatic eviction	TTL-based automatic eviction	Instant (TTL aligned with `expires_at`)
Cache Update (counter)	Not cached in L1 (counters are async)	Periodic bulk refresh from analytics pipeline	1-5 minutes

Invalidation Flow for Deletion:

```
1. Application Server receives DELETE request
2. Write tombstone to Cassandra (is_deleted = TRUE)
3. DEL key from local region's Redis (L2)
4. Publish invalidation message to Redis Pub/Sub channel: "invalidate:{token}"
5. All pods in all regions receive Pub/Sub message
6. Each pod evicts the token from its L1 cache
7. Each region's Redis subscriber evicts the token from its L2 cache
```

---

14. Failure Behavior & Operational Signals

14.1 Failure Scenario Analysis

Scenario F1: ZooKeeper Cluster Failure

Aspect	Detail
Impact	TGS workers cannot acquire new token ranges
Blast Radius	Write path only; read path completely unaffected
Time to Impact	Workers continue operating on their current range. At average load, each worker has ~7 hours of token capacity remaining.
Degradation	None immediate. Graceful degradation begins when the first worker exhausts its range.
Detection	ZK session timeout alerts (heartbeat failure within 10 seconds). Worker-level metric: `tgs_range_remaining_tokens` drops below threshold.
Mitigation	Workers pre-fetch next range at 80% consumption. Emergency fallback: switch to UUID-based token generation (longer tokens, but zero coordination needed).
Recovery	ZK cluster self-heals via leader election. Workers resume range acquisition on reconnection.

Scenario F2: Redis Cache Eviction Storm

Aspect	Detail
Impact	Sudden spike in cache misses; all traffic falls through to Cassandra
Blast Radius	Regional; the affected region's L2 cache is compromised
Time to Impact	Immediate; p99 read latency spikes from 5ms to 50-100ms
Detection	`redis_evicted_keys_total` counter spike. `cache_hit_ratio` drops below 80%. Cassandra `read_latency_p99` increases.
Mitigation	Circuit breaker on Cassandra: if read latency exceeds 200ms, return stale cached values (serve from L1 even if expired). Auto-scaling provisions additional Redis nodes. Rate-limit non-critical reads.
Recovery	Cache self-warms as traffic naturally repopulates entries. Proactive warming script reads top-1M tokens from Cassandra and pre-loads Redis.

Scenario F3: Cross-Region Replication Lag Spike

Aspect	Detail
Impact	URLs created in one region are not yet visible in other regions
Blast Radius	Affects only newly created URLs (seconds-old); established URLs are unaffected
Time to Impact	Proportional to lag duration (typically seconds, worst case minutes)
Detection	`cassandra_cross_dc_replication_lag_seconds` metric exceeds threshold. Nagios/PagerDuty alert at > 30 seconds lag.
Mitigation	Read-your-own-write guarantee via session-level routing: the creating client's subsequent reads are routed to the same region. Cross-region 404s for brand-new URLs are an accepted trade-off (see Section 5).
Recovery	Cassandra's anti-entropy repair process and hinted handoff mechanism ensure eventual convergence.

Scenario F4: Complete Region Outage

Aspect	Detail
Impact	One-third of global capacity offline
Time to Impact	Anycast DNS health check detects failure within 10-30 seconds
Failover	DNS automatically routes traffic to remaining healthy regions. Cassandra's `NetworkTopologyStrategy` ensures full data availability from surviving DCs.
Capacity	Remaining 2 regions must absorb 50% more traffic each. Auto-scaling provisions additional pods within 60 seconds.
Recovery	Failed region comes back online; Cassandra streams missing data via repair. DNS gradually reintroduces the region via weighted routing.

14.2 Critical Observability Metrics

RED Metrics (Rate, Errors, Duration)

Metric	Instrument	Alert Threshold
`redirect_requests_per_second`	Counter	Warning: > 80% of design target (160K/s). Critical: > 95% (190K/s).
`redirect_error_rate`	Ratio (5xx / total)	Warning: > 0.01%. Critical: > 0.1%.
`redirect_latency_p99`	Histogram	Warning: > 10ms. Critical: > 50ms.
`shorten_requests_per_second`	Counter	Warning: > 300/s. Critical: > 380/s.
`shorten_error_rate`	Ratio (5xx / total)	Warning: > 0.1%. Critical: > 1%.
`shorten_latency_p99`	Histogram	Warning: > 80ms. Critical: > 200ms.

Infrastructure Health Signals

Signal	Source	Alert Condition
`cassandra_read_latency_p99`	Cassandra JMX	> 50ms
`cassandra_write_latency_p99`	Cassandra JMX	> 100ms
`cassandra_pending_compactions`	Cassandra JMX	> 50
`redis_connected_clients`	Redis INFO	> 80% of maxclients
`redis_memory_usage_ratio`	Redis INFO	> 85% of maxmemory
`redis_cache_hit_ratio`	Custom	< 85%
`kafka_consumer_lag`	Kafka metrics	> 100,000 events
`zk_outstanding_requests`	ZK JMX	> 100
`tgs_range_remaining_pct`	App metric	< 20%
`dns_health_check_failures`	Route53/CF	Any failure in 2+ consecutive checks

SLO Monitoring (Burn Rate Alerts)

```
SLO: 99.999% redirect availability (5.26 min/year error budget)

Burn Rate Alert Configuration:
  - 14.4x burn rate over 1 hour  → PAGE (will exhaust budget in 2.5 days)
  - 6x burn rate over 6 hours    → PAGE (will exhaust budget in 6 days)
  - 3x burn rate over 1 day      → TICKET (will exhaust budget in 10 days)
  - 1x burn rate over 3 days     → REVIEW (on track to exhaust budget)

Error Budget Remaining = 1 - (total_error_minutes_ytd / 5.26)
```

14.3 Runtime Health-Check Endpoints

```
GET /healthz          → 200 OK (liveness: process is running)
GET /readyz           → 200 OK (readiness: can serve traffic)
                        503 if: DB connection pool exhausted,
                                Redis unreachable,
                                TGS range < 1% remaining
GET /healthz/deep     → 200 OK with JSON body:
{
  "status": "HEALTHY",
  "checks": {
    "cassandra": {"status": "UP", "latency_ms": 2.3},
    "redis": {"status": "UP", "latency_ms": 0.8, "hit_ratio": 0.94},
    "kafka": {"status": "UP", "lag": 1203},
    "tgs": {"status": "UP", "range_remaining_pct": 67.2},
    "memory": {"heap_used_pct": 43.1, "gc_pause_p99_ms": 12}
  },
  "region": "NA-East",
  "version": "v2.14.3",
  "uptime_seconds": 1482937
}
```

15. Interview Walkthrough Decision Summary & Checklist

15.1 Trade-Off Decision Cheat Sheet

- **AP over CP:** Availability and partition tolerance prioritized. Eventual consistency accepted for reads. Justification: read path is revenue-critical; temporary 404 for unreplicated new URLs is acceptable; token uniqueness guaranteed by construction (TGS ranges), not by consensus.
- **Cassandra over PostgreSQL:** AP-native, multi-DC replication built-in, LSM-tree optimized for write-heavy append workloads, linear horizontal scaling via consistent hashing. Trade-off: no joins, no multi-row transactions — acceptable because the access pattern is pure key-value point lookups.

- **Base62 over Base64/UUID:** 7-character tokens are URL-safe, human-readable, and typeable. 62^7 = 3.52 trillion keyspace provides centuries of capacity at stated scale. Trade-off: slightly larger keyspace needed vs. Base64, but eliminates URL-encoding issues with `+`, `/`, `=`.
- **Range-Based TGS over Random Generation:** Zero-collision guarantee by construction eliminates read-before-write checks. Trade-off: ZooKeeper dependency for range allocation; mitigated by pre-fetching and 7-hour range buffers.
- **Consistent Hashing over Range/Modular Sharding:** Minimal data movement on rebalancing (1/N keys). Native Cassandra support. Trade-off: no efficient range scans — acceptable because primary access is point-reads.
- **Multi-Tier Cache (L1+L2) over Single Cache Layer:** L1 (in-process) provides sub-millisecond lookups without network hop. L2 (Redis) provides shared state across pods. Trade-off: cache coherence complexity; mitigated by Pub/Sub invalidation and short TTLs.
- **Kafka Analytics Pipeline over Synchronous Writes:** Decouples analytics from redirect hot path. Trade-off: analytics are eventually consistent (seconds delay); acceptable because real-time click counts are not a business requirement.
- **302 (Temporary) over 301 (Permanent) Redirect:** 302 allows the server to see every click for analytics. 301 would cause browsers to cache the redirect permanently, bypassing analytics. Trade-off: slightly higher server load; justified by analytics requirement.
- **Anycast DNS over Geo-DNS:** Anycast routes at the network layer (BGP), not the application layer. Provides automatic failover without DNS TTL propagation delay. Trade-off: less control over traffic distribution; mitigated by health-checked failover.
- **Soft-Delete over Hard-Delete:** Preserves analytics integrity and enables audit trails. Trade-off: storage cost of tombstones; mitigated by background compaction after analytics retention window (90 days).
- **LRU with XFetch over Simple TTL Expiration:** XFetch probabilistically refreshes cache entries before expiry, preventing stampedes. Trade-off: slightly more complex cache logic; justified by the 200K reads/sec design target.
- **Fire-and-Forget Kafka Producer over Guaranteed Delivery:** `acks=1` with async batching maximizes throughput on the redirect hot path. Trade-off: < 0.01% event loss probability; acceptable for analytics (not financial data).

15.2 10-Point Architectural Checklist

Before concluding a system design interview for a URL shortener (or similar high-read system), verify that every item below has been explicitly addressed:

- [ ] **1. Read/Write Asymmetry Acknowledged:** The design explicitly treats reads and writes as separate planes with different optimization targets, consistency requirements, and scaling strategies.
- [ ] **2. Token Uniqueness Mechanism Defined:** The token generation strategy guarantees global uniqueness without runtime coordination. The mechanism (range pre-allocation, consensus cluster, failure recovery) is fully specified.

- [ ] **3. CAP Position Stated and Justified:** The design explicitly chooses AP or CP, with concrete failure-mode analysis showing why the chosen trade-off is acceptable for this workload.
- [ ] **4. Capacity Math is Transparent:** QPS (read/write), bandwidth, storage, and cache sizing are derived from first principles with stated assumptions. Peak/burst factors and design headroom are explicit.
- [ ] **5. Multi-Tier Caching Strategy Present:** The design includes at least L1 (in-process) and L2 (distributed) cache layers, with explicit eviction policies, TTLs, invalidation mechanics, and cache stampede protection.
- [ ] **6. Sharding Strategy Specified:** The data partitioning approach is named (consistent hashing), justified against alternatives, and the partition key is explicitly identified.
- [ ] **7. Analytics Decoupled from Hot Path:** Click event capture does not add synchronous latency to the redirect response. The analytics pipeline (Kafka/Flink/ClickHouse) is architecturally separate.
- [ ] **8. Failure Modes Analyzed:** At least 3 specific infrastructure failures are analyzed with their blast radius, detection mechanism, mitigation strategy, and recovery procedure.
- [ ] **9. Global Distribution Addressed:** The design includes multi-region deployment with Anycast/Geo DNS, cross-region replication, and automatic failover. Single-region failure does not cause global outage.
- [ ] **10. API Contract Complete:** Request/response formats, HTTP status codes, cache-control headers, rate-limiting headers, and idempotency strategies are fully specified for all core endpoints.

End of Document

Document Hash (SHA-256): `To be computed upon final compilation`
Classification: Public — Interview Preparation Material
License: CC BY-SA 4.0